



UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Nombre de la Organización:

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO (UTEQ)

Presenta:

VICTOR GUADALUPE LOPEZ CEBREROS

Grupo:

IDGS17

Práctica 4. Herramientas para Administración de Logs. Parte 1.

SEGG-U2-P4H-1 — Plataforma de Observabilidad con Grafana, Loki y Promtail

1. Datos generales

Campo	Valor
Aplicación monitoreada	VulnerableApp, ejecutándose localmente con dotnet run en http://localhost:5277
Infraestructura desplegada	Grafana, Loki y Promtail en contenedores Docker
Interfaz de Grafana	http://localhost:3000
Endpoint interno de Loki	http://loki:3100
Origen de los registros	VulnerableApp/Logs/*.txt, montados en solo lectura
Red de contenedores	observability
Volúmenes persistentes	grafana_data, loki_data

2. Objetivo

Incorporar una plataforma de observabilidad sobre una aplicación en ejecución **sin modificar su código ni containerizarla**, recolectando los registros que ya produce mediante un agente externo, indexándolos por etiquetas y consultándolos a través de un lenguaje de consulta especializado.

El planteamiento es en sí mismo el contenido de la práctica: demostrar que las capacidades de monitoreo pueden agregarse de forma desacoplada del ciclo de vida del código de negocio.

3. Arquitectura implementada

3.1 Componentes y responsabilidades

Componente Responsabilidad

Promtail	Agente de recolección. Lee los archivos de log del sistema de archivos, aplica transformaciones sobre cada línea y envía los eventos a Loki.
Loki	Motor de almacenamiento e indexación. Indexa exclusivamente las etiquetas de cada flujo, conservando el cuerpo del mensaje comprimido sin indexar.
Grafana	Capa de visualización y consulta. Ejecuta consultas LogQL contra Loki y presenta los resultados en vistas exploratorias o paneles.

Estructura de la carpeta de infraestructura:

observability-infra/

```
|— docker-compose.yml
|— grafana/
|  └— provisioning/datasources/datasource.yml
└— promtail/
    └— promtail-config.yml
```

3.2 Decisiones de configuración

Bind mount de solo lectura. El directorio de registros se monta en el contenedor de Promtail mediante `../VulnerableApp/Logs:/var/log/vulnerableapp:ro`. El sufijo `:ro` es una decisión de seguridad deliberada: Promtail únicamente necesita leer los archivos, nunca escribirlos, de modo que restringir el permiso impide que un compromiso del contenedor derive en la alteración o el borrado de la evidencia. Los registros constituyen precisamente el artefacto que un atacante buscaría manipular para encubrir su actividad, por lo que protegerlos frente al propio sistema que los recolecta es coherente con el principio de privilegio mínimo.

Aprovisionamiento declarativo del origen de datos. La conexión de Grafana con Loki se define en un archivo de configuración bajo `grafana/provisioning/datasources/`, en lugar de crearse manualmente desde la interfaz. Esto convierte la configuración en código versionable y reproducible:

cualquier despliegue nuevo del entorno reproduce el mismo estado sin intervención manual ni pasos susceptibles de olvido.

Supresión de la clave version. Se eliminó del archivo de composición, dado que Compose v2 la considera obsoleta y emite una advertencia al procesarla.

No containerización de la aplicación. VulnerableApp continúa ejecutándose de forma local, tal como en las prácticas previas. La comunicación entre la aplicación y la plataforma ocurre exclusivamente a través del sistema de archivos, sin acoplamiento de red ni dependencia de arranque entre ambos.

4. Procesamiento en Promtail

Se configuró un pipeline_stages de cuatro etapas que transforma cada línea de texto en un evento con etiquetas consultables, **sin modificar una sola línea del código de la aplicación.**

Etapas	Función
regex	Parsea el formato de salida de Serilog (timestamp [LEVEL] [cid:...] mensaje) y extrae los campos timestamp, level, correlation_id y message.
template	Evalúa el contenido de message mediante expresiones regulares y calcula el campo category.
labels	Promueve level y category de campos extraídos a etiquetas reales de Loki.
timestamp	Instruye a Loki para que emplee la marca temporal contenida en el registro en lugar de la hora de ingesta.

La última etapa merece atención particular. Sin ella, todo evento quedaría fechado con el momento en que Promtail lo leyó, no con aquel en que ocurrió. Ante un reinicio del agente o un retraso en la recolección, el orden cronológico se distorsionaría y cualquier correlación temporal con otras fuentes resultaría inválida.

4.1 Clasificación por categoría

El campo category asigna a cada línea uno de tres valores:

Categoría	Contenido
security	Eventos de Auth.*, AuthEvent:* y verificación de hash BCrypt

Categoría	Contenido
-----------	-----------

audit	Eventos de Comment.*, Api.* y operaciones sobre la base de datos (Executed DbCommand, INSERT, UPDATE, DELETE)
-------	---

vulnerableapp El resto: página principal, búsquedas y actividad general

Esta segmentación se realiza **en el agente de recolección y no en la aplicación**. La consecuencia práctica es que la separación de dominios —seguridad, auditoría y operación general— se obtiene sin que la aplicación deba escribir a archivos distintos ni conocer la existencia de la clasificación. Si mañana se requiriera una cuarta categoría, el cambio se limita al archivo de configuración de Promtail.

4.2 Criterio de cardinalidad

El `correlation_id` se conservó deliberadamente como **campo extraído y no como etiqueta**. La razón responde al modelo de indexación de Loki: cada combinación única de etiquetas constituye un flujo independiente, por lo que promover a etiqueta un valor que cambia en cada petición generaría tantos flujos como peticiones atendidas, degradando gravemente el rendimiento del índice y el consumo de memoria del sistema.

Este criterio —conocido como control de cardinalidad— constituye la regla de diseño fundamental al configurar Loki: las etiquetas deben contener valores de dominio acotado y previsible, como el nivel de severidad o la categoría, mientras que los identificadores únicos se recuperan mediante filtros de línea sobre el cuerpo del mensaje.

5. Consultas LogQL implementadas

logql

Todos los eventos de la aplicación

```
{job="vulnerableapp"}
```

Eventos de autenticación y seguridad

```
{job="vulnerableapp", category="security"}
```

Operaciones auditables: comentarios, API y base de datos

```
{job="vulnerableapp", category="audit"}
```

Actividad general restante

```
{job="vulnerableapp", category="vulnerableapp"}
```

Errores ocurridos durante operaciones auditables

```
{job="vulnerableapp", category="audit", level="ERR"}
```

La última consulta ilustra el valor de la combinación de etiquetas: responde a una pregunta específica —qué falló durante operaciones sobre datos— que sobre un archivo plano exigiría dos filtros textuales encadenados sobre el volumen completo del registro.

6. Actividad generada para la verificación

Se ejercitó la aplicación cubriendo las categorías de evento requeridas: navegación general, búsqueda, autenticación fallida y exitosa, acceso al panel, cierre de sesión, alta de comentario válido y vacío, consultas a la API, excepción controlada y excepción no controlada.

La secuencia se diseñó para producir eventos de los tres niveles de severidad relevantes y de las tres categorías configuradas, de modo que la verificación de las consultas por etiqueta pudiera realizarse sobre datos representativos y no sobre un registro homogéneo.

7. Análisis comparativo: Seq frente a Grafana con Loki

Característica	Seq	Grafana + Loki
Búsqueda	Motor propio orientado a propiedades estructuradas de Serilog, con sintaxis próxima a SQL. Muy directo para logs de .NET.	LogQL filtra primero por etiquetas indexadas y después aplica filtros de línea sobre el resultado acotado. Menos inmediato, considerablemente más potente al combinar ambos niveles.
Dashboards	Limitados. Seq es fundamentalmente un visor y buscador de eventos.	Punto fuerte de Grafana: paneles configurables, gráficas de tasa de error y tableros que combinan múltiples orígenes de datos.
Alertas	Alertas básicas sobre consultas guardadas.	Reglas flexibles con múltiples canales de notificación y umbrales

Característica Seq

Grafana + Loki

		sobre series temporales derivadas de los registros.
Lenguaje de consulta	Orientado a propiedades estructuradas.	LogQL, inspirado en PromQL: selección por etiquetas, filtros de línea y funciones de agregación como <code>rate()</code> y <code>count_over_time()</code> .
Visualización	Lista de eventos expandibles, muy legible para depurar un caso concreto.	Paneles de series temporales, tablas y vistas de log, orientados a tendencias y correlación.
Uso principal	Depuración puntual durante el desarrollo: «¿qué ocurrió en esta petición?».	Observabilidad de plataforma: monitoreo continuo, correlación con métricas y trazas, alertamiento en producción.

La conclusión del comparativo no es que una herramienta supere a la otra, sino que responden a preguntas de naturaleza distinta. Seq resulta superior para investigar un incidente concreto ya identificado; Grafana con Loki resulta superior para **detectar** que existe un incidente, mediante la observación de tendencias agregadas. En un entorno real ambas coexisten con frecuencia, y de hecho en esta práctica ambas consumen la misma fuente de registros sin interferencia mutua.

8. Respuestas al reto

8.1 ¿Qué modificaciones se realizaron en Promtail?

Se incorporó un `pipeline_stages` de cuatro etapas conforme al detalle de la sección 4: parseo de la línea con regex, cálculo del campo `category` mediante template con evaluación de expresiones regulares sobre el mensaje, promoción de `level` y `category` a etiquetas con `labels`, y ajuste de la marca temporal con `timestamp`.

Ninguna de estas modificaciones tocó el código de la aplicación.

8.2 ¿Cómo utiliza Loki las etiquetas?

Loki indexa **únicamente las etiquetas**, no el texto completo de cada línea, lo cual constituye su diferencia arquitectónica esencial frente a motores como Elasticsearch. Cada combinación única de etiquetas conforma un flujo independiente, y el índice resultante es pequeño y económico de mantener.

Al ejecutar una consulta como `{job="vulnerableapp", category="audit"}`, Loki emplea ese índice para localizar directamente los flujos pertinentes, sin recorrer el resto del almacenamiento. Solo entonces, y de forma opcional, aplica los filtros de texto sobre las líneas ya acotadas, operación que se realiza mediante lectura secuencial pero sobre un volumen sustancialmente menor.

Esta arquitectura explica tanto el bajo costo de almacenamiento de Loki como la importancia crítica del control de cardinalidad descrito en la sección 4.2: el diseño resulta eficiente mientras el número de flujos permanezca acotado, y se degrada con rapidez cuando no lo está.

8.3 ¿Qué ventajas ofrecen las etiquetas para consultar con LogQL?

- **Rendimiento.** El filtrado inicial por etiqueta es indexado, a diferencia del recorrido secuencial del texto completo.
- **Separación de dominios.** Permiten aislar seguridad, auditoría y operación general dentro de un mismo archivo de log, sin que la aplicación escriba a destinos separados ni conozca la clasificación.
- **Composición.** Varias etiquetas se combinan en una sola consulta — `category="audit" junto con level="ERR"`— para responder preguntas específicas.
- **Agregación.** Habilitan funciones como `count_over_time` y `rate` agrupadas por etiqueta, base de dashboards y alertas. La tasa de eventos con `category="security" y level="WRN"` funciona, por ejemplo, como indicador directo de intentos de autenticación fallidos, y sobre ella puede definirse una alerta por umbral.

9. Pregunta de reflexión

¿Qué ventajas ofrece incorporar una plataforma de observabilidad sin modificar la aplicación existente? ¿Cómo facilita esta arquitectura la evolución de un sistema en producción?

La incorporación de Grafana, Loki y Promtail sin intervenir el código de VulnerableApp materializa el principio de **separación de responsabilidades**. La aplicación asume una única obligación: producir registros de calidad, estructurados, con identificador de correlación y con el nivel de severidad adecuado. La plataforma de observabilidad asume el resto: recolectar, transformar, indexar, consultar y visualizar.

La primera ventaja es la **reducción del riesgo operativo**. Ganar capacidades de monitoreo no exige recompilar, volver a probar ni desplegar la aplicación. En un entorno productivo esto se traduce en ausencia de interrupción del servicio y en la

eliminación del riesgo de introducir un defecto nuevo por el solo hecho de agregar instrumentación. Promtail opera como un adaptador externo que lee archivos preexistentes mediante un montaje del sistema de archivos, de modo que la fuente de verdad permanece inalterada.

La segunda ventaja es la **flexibilidad ante la evolución del sistema**. Una eventual migración de la aplicación a contenedores o a un orquestador, o un cambio en el formato de sus registros, se absorbe ajustando la configuración de Promtail —concretamente sus `pipeline_stages`— o reapuntando el agente al nuevo destino, sin que el equipo de desarrollo deba intervenir. La observabilidad evoluciona a su propio ritmo, desacoplada del ciclo de vida del código de negocio.

La tercera ventaja es la **construcción incremental**. Al mismo Grafana pueden incorporarse posteriormente métricas mediante Prometheus y trazas distribuidas mediante Tempo, sin volver a tocar la aplicación. Cada señal se agrega cuando resulta necesaria, y todas convergen en una única superficie de consulta que permite correlacionarlas entre sí.

Existe, no obstante, una contrapartida que conviene reconocer. La instrumentación externa está limitada por lo que la aplicación efectivamente registra: si un evento no se emite, ninguna plataforma podrá recuperarlo. La observabilidad desacoplada es una capa de explotación, no un sustituto de una instrumentación adecuada en el código. De ahí que las prácticas previas —donde se incorporó Serilog, el identificador de correlación y la instrumentación de los controladores— constituyan el requisito habilitante de esta.

10. Conclusiones

Se desplegó una plataforma de observabilidad completa sobre una aplicación en ejecución sin modificar su código ni su forma de despliegue, comunicándose ambas exclusivamente a través del sistema de archivos.

El procesamiento configurado en Promtail demuestra que la estructura útil para el análisis puede construirse en la capa de recolección: la clasificación en categorías de seguridad, auditoría y operación general se obtuvo sin que la aplicación escriba a destinos separados ni tenga conocimiento alguno de esa taxonomía.

El modelo de indexación de Loki, restringido a las etiquetas, impone una disciplina de diseño explícita respecto de qué información se promueve a etiqueta y cuál permanece como contenido del mensaje. La decisión de conservar el identificador de correlación como campo extraído ilustra ese criterio: se trata precisamente del dato de mayor cardinalidad y, por tanto, del peor candidato posible a etiqueta.

Finalmente, la coexistencia de Seq y de Grafana con Loki sobre la misma fuente de registros evidencia que ambas herramientas son complementarias antes que

sustitutas, y que una arquitectura de observabilidad desacoplada admite añadir consumidores sin costo alguno para la aplicación observada.

11. evidencias

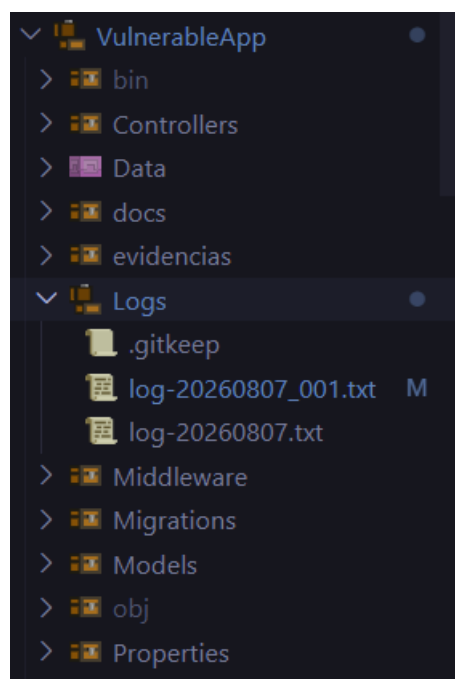
```
C:\Users\victo\Documents\Práctica Creación de Solución\observability-infra>docker compose up -d
[+] up 3/3
✓ Container loki      Running
✓ Container grafana   Running
✓ Container promtail  Running

What's next:
  Filter, search, and stream logs from all your Compose services
  in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs?appId=observability-infra

C:\Users\victo\Documents\Práctica Creación de Solución\observability-infra>docker compose ps
NAME          IMAGE              COMMAND                  SERVICE    CREATED          STATUS          PORTS
grafana       grafana/grafana:10.4.3  "/run.sh"               grafana    30 minutes ago  Up 30 minutes  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
loki          grafana/loki:2.9.8    "/usr/bin/loki -conf..." loki       30 minutes ago  Up 30 minutes  0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp
promtail     grafana/promtail:2.9.8  "/usr/bin/promtail -..." promtail    30 minutes ago  Up 30 minutes

C:\Users\victo\Documents\Práctica Creación de Solución\observability-infra>
```

```
C:\Users\victo\Documents\Práctica Creación de Solución\VulnerableApp>dotnet run
Usando la configuración de inicio de C:\Users\victo\Documents\Práctica Creación de Solución\VulnerableApp\Properties
Compilando...
C:\Users\victo\Documents\Práctica Creación de Solución\VulnerableApp\Models\User.cs(6,23): warning CS8618: El elemento
  ULL debe contener un valor distinto de NULL al salir del constructor. Considere la posibilidad de agregar el modificador
  valor que acepta valores NULL.
C:\Users\victo\Documents\Práctica Creación de Solución\VulnerableApp\Models\User.cs(7,23): warning CS8618: El elemento
  es NULL debe contener un valor distinto de NULL al salir del constructor. Considere la posibilidad de agregar el modificador
  o un valor que acepta valores NULL.
C:\Users\victo\Documents\Práctica Creación de Solución\VulnerableApp\Models\User.cs(8,23): warning CS8618: El elemento
  debe contener un valor distinto de NULL al salir del constructor. Considere la posibilidad de agregar el modificador
  lor que acepta valores NULL.
[16:24:30 INF] Iniciando VulnerableApp...
[16:24:32 INF] [cid:] Now listening on: http://localhost:5277
[16:24:32 INF] [cid:] Application started. Press Ctrl+C to shut down.
[16:24:32 INF] [cid:] Hosting environment: Development
[16:24:32 INF] [cid:] Content root path: C:\Users\victo\Documents\Práctica Creación de Solución\VulnerableApp
```



localhost:5341/#/events?range=1d

Render Dashboard

Seq Personal Find commands, shortcuts, and screens Ctrl+K Events Metrics Dashboards

Content root path: C:\Users\Victo\Documents\Práctica Creación de Solución\VulnerableApp
Hosting environment: Development
Application started. Press Ctrl+C to shut down.
Now listening on: http://localhost:5277
Content root path: C:\Users\Victo\Documents\practicass\VulnerableApp
Hosting environment: Development
Application started. Press Ctrl+C to shut down.
Now listening on: http://localhost:5277

- VulnerableApp termino inesperadamente durante el arranque
- Hosting failed to start

HTTP GET / respondio 200 en 28.5086 ms
Fin Home.Index Usuario:anonimo DuracionMs:6
Inicio Home.Index Usuario:anonimo IP::1
HTTP GET / respondio 200 en 20.3944 ms
Fin Home.Index Usuario:anonimo DuracionMs:3
Inicio Home.Index Usuario:anonimo IP::1
HTTP GET /Auth/Dashboard respondio 200 en 2.6801 ms
Fin Auth.Dashboard Usuario:user1 DuracionMs:2
Executed DbCommand (2ms) [Parameters=[@p=? (DbType = Int32)], CommandType=Text, CommandTimeout=30] SELECT TOP(1) [u].[Id], [u...]
Inicio Auth.Dashboard Usuario:user1 IP::1
HTTP GET /Auth/Dashboard respondio 200 en 3.8116 ms
Fin Auth.Dashboard Usuario:admin DuracionMs:2
Executed DbCommand (0ms) [Parameters=[@p=? (DbType = Int32)], CommandType=Text, CommandTimeout=30] SELECT TOP(1) [u].[Id], [u...]
Inicio Auth.Dashboard Usuario:admin IP::1
HTTP GET /Auth/Dashboard respondio 200 en 3.3882 ms
Fin Auth.Dashboard Usuario:user1 DuracionMs:2
Executed DbCommand (2ms) [Parameters=[@p=? (DbType = Int32)], CommandType=Text, CommandTimeout=30] SELECT TOP(1) [u].[Id], [u...]
Inicio Auth.Dashboard Usuario:user1 IP::1
HTTP GET /Auth/Dashboard respondio 200 en 4.1158 ms

localhost:3000/connections/datasources/edit/P8E80F9AEF21F6940

Render Dashboard

Home > Connections > Data sources > Loki

Alerting
Manage alert rules for the Loki data source. [Learn more about alerting](#)

Manage alert rules in Alerting UI ☐

Queries
Additional options to customize your querying experience. [Learn more about query settings](#)

Maximum lines 1000

Derived fields
Derived fields can be used to extract new fields from a log message and create a link from its value. [Learn more about derived fields](#)

+ Add

✓ Data source successfully connected.
Next, you can start to visualize data by [building a dashboard](#), or by querying data in the [Explore](#) view.

Delete Save & test

Render Dashboard

localhost:3000/explore

Search or jump to... ctrl+k

Home > Explore

Home

Starred

Dashboards

Explore

Alerting

Connections

Add new connection

Data sources

Administration

Outline

Loki

Split

Add to dashboard

Builder

Code

Live

Kick start your query

Label browser

Explain query

Label filters

level = ERR

1 {level="ERR"}

Fetch all log lines matching label filters.

Line contains

ogql # Todo lo que generó la app (job="vulnerableapp") # Solo event

2 <expr> |= ogql # Todo lo que generó la app (job="vulnerableapp") # Solo eventos de autenticación (login, logout, dashboard, verificación BCrypt) {job="vulnerableapp", level="ERR"}

Return log lines that contain string

ogql # Todo lo que generó la app (job="vulnerableapp") # Solo eventos de autenticación (login, logout, dashboard, verificación BCrypt) {job="vulnerableapp", level="ERR"}

{level="ERR"} |= `ogql # Todo lo que generó la app (job="vulnerableapp") # Solo eventos de autenticación (login, logout, dashboard, verificación BCrypt) {job="vulnerableapp", level="ERR"}`

Options

Type: Range

Line limit: 1000